# CSC5041: Quiz 1 Revision Sheet

## 1. Database systems and the three-schema architecture

### The database approach

- A **DBMS** stores the data *and* its description (*metadata*), controls access and integrity, and hides how the data is physically stored.
- **schema** = the description of the data (changes rarely); **instance** = the data actually stored at one moment (changes constantly).

### Three-schema (three-level) architecture

- *external level* — the views that particular user groups are allowed to see.
- *conceptual level* — the logical description of the whole database: entities, attributes, domains, integrity rules.
- *internal level* — how the data is stored: files, access paths, indexes.
- the two *mappings* — external/conceptual and conceptual/internal.

### Data independence

- *logical* — a change to the conceptual schema need not force application programs to change (views shield them).
- *physical* — a change to the internal schema (new disks, moved files, rebuilt indexes) leaves the conceptual schema and the programs untouched.

## 2. Conceptual design: the ER model

### Building blocks

- **entity type** — a class of objects of the same kind (a rectangle); an **instance** is one object of that class.
- **attribute** — a property of an entity or of a relationship (an ellipse); **relationship type** — an association between entity types (a diamond).

### Kinds of attribute and of key

- attribute: *simple* (atomic) · *composite* (has parts: `address` = street, city) · *multivalued* (a set) · *derived* (computable, e.g. `age` from a date of birth).
- key: *superkey* (determines a tuple) ⊃ *candidate key* (minimal) ⊃ *primary key* (the chosen one).
- *partial key* — identifies the instances of a weak entity *within one owner*.

### Three things to determine for every relationship

- *degree* — how many entity types take part: unary (recursive), binary, ternary.
- *cardinality ratio* — 1:1, 1:$N$, $M$:$N$, written next to the relationship, one number per participating side.
- *participation* — *total* (every instance must take part, double line) or *partial* (single line).

### Weak entity

- No key of its own; identified by an *owner* through an *identifying relationship*; its partial key is unique only inside one owner.

### Notation

- rectangle = entity; **double rectangle** = weak entity; diamond = relationship; **double diamond** = identifying relationship.
- ellipse = attribute; **double ellipse** = multivalued; *dashed ellipse* = derived (not stored).
- underline = key; *dashed underline* = partial key; **double line** to an entity = total participation.

## 3. The relational model and ER → relational mapping

### Relational model

- A *relation* is a set of tuples over a fixed set of attributes; values are atomic and the order of tuples does not matter.

### Four integrity constraints

- *domain* — every value comes from the attribute's domain.
- *key* — a candidate key identifies at most one tuple (unique).
- *entity integrity* — a primary key is never null.
- *referential integrity* — a foreign key matches an existing key, or is wholly null.

### Mapping rules

- **strong entity** → a relation; its key → the primary key.
- **composite** attribute → flatten into its simple components.
- **multivalued** attribute → its own relation: owner key + attribute together form the primary key.
- **weak entity** → owner key + partial key = primary key; the owner key is also a foreign key.
- 1**:**1 → one side takes a *unique* foreign key (preferably the total side); a separate relation is also acceptable.
- 1**:**$N$ → the "one" side's key becomes a foreign key on the "many" side.
- $M$**:**$N$ → a new relation whose primary key is both keys, plus any attributes of the relationship.
- $n$**-ary** → a new relation over all participants; the primary key is the keys of the sides of cardinality $N$, while a key of a 1-side is only a foreign key.
- **derived** attribute → never stored.

## 4. Relational algebra

### Operators

- $\sigma_{cond}(R)$ *selection* — keep the tuples that satisfy *cond*.
- $\pi_{A,B}(R)$ *projection* — keep the listed attributes, duplicates removed.
- $\rho_S(R)$ *rename* — rename a relation (or its attributes).
- $R \times S$ *product*; $R \cup S$, $R \cap S$, $R - S$ *set operations* (union-compatible operands).
- $R \bowtie_{cond} S$ *theta join*; $R \bowtie S$ *natural join*; $R \div S$ *division*.
- $\gamma_{grp;agg}(R)$ *aggregation* — grouping plus count, sum, avg, min, max.

### Patterns worth remembering

- "all / every / both" → *division*: build the pair table, divide by the required set, then project.
- "none / never / no" → *set difference*.
- "more than one", or comparing two rows of one relation → rename the relation twice with $\rho$ and join the copies on a $\neq$ condition.
- "largest / most" → *aggregation*, then compare with the extremum.
- "how many" → aggregation with `count(distinct ...)`, never a projection alone.

### Easy mistakes

- $\pi$ eliminates duplicates, so it is not a counting operator.
- "below 100" is strict: use $<$, not $\leq$.
- Join on keys, never on names or titles; and remember a join multiplies rows, so count what the question really asks for.